A 22-nm 109.3-to-249.5-TFLOPS/W Outlier-Aware Floating-Point SRAM Compute-in-Memory Macro for Large Language Models

Siqi He, *Graduate Student Member, IEEE*, Haozhe Zhu, *Member, IEEE*, Hongyi Zhang, *Graduate Student Member, IEEE*, Yujie Ma, Zexing Chen, Mengjie Li, *Graduate Student Member, IEEE*, Danfeng Zhai, *Graduate Student Member, IEEE*, Chixiao Chen, *Member, IEEE*, Qi Liu, *Member, IEEE*, Xiaoyang Zeng, *Member, IEEE*, and Ming Liu, *Fellow, IEEE*

***Abstract*—Large language models (LLMs) have demonstrated exceptional performance in complex artificial intelligence (AI) tasks. However, their rapidly increasing parameter sizes lead to significant communication and computational overhead, posing challenges to the energy efficiency (EEF) and memory footprint of AI processors. Compute-in-memory (CIM) architecture has emerged as a promising solution to alleviate bandwidth constraints and improve EEF. Nonetheless, both integer (INT) and floating-point (FP) CIM implementations struggle with the trade-off between accuracy and memory requirement when applied to LLMs. Outlier-aware quantization (OAQ), which employs low-precision formats for normal values and retains FP formats for high-magnitude outliers, has proven effective in matching the accuracy of full-FP baselines and has become a mainstream approach for efficient LLM deployment. Therefore, this work presents OA-CIM, an SRAM-based digital CIM macro that facilitates element-wise hybrid processing of BF16 outliers and INT4 normal values. The major contributions are: 1) an LUT-based multiply-and-accumulate (MAC) circuit design, which supports efficient FP/INT-compatible (FIC) MAC operations; 2) an XOR-sharing non-maximum exponent gating scheme that reduces latency and area by bypassing unnecessary exponent comparisons in FP dataflow; and 3) a sparsity-aware readout circuit with distribution-offset weight encoding (DOWE) to mitigate the power-intensive charging/discharging process on the bitline. A 22-nm 512-kB 8T SRAM OA-CIM prototype is fabricated, which achieves an EEF of 346.6 TOPS/W in INT4 mode and 249.5 TFLOPS/W in outlier mode, representing a 2.7× to 3.1× improvement over state-of-the-art mixed-precision CIMs.**

***Index Terms*—Artificial intelligence (AI), compute-in-memory (CIM), large language models (LLMs), lookup table, outlier-aware quantization (OAQ).**

Received 4 January 2025; revised 10 March 2025 and 29 April 2025; accepted 12 May 2025. Date of publication 28 May 2025; date of current version 30 January 2026. This article was approved by Associate Editor Ben Keller. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62304047 and Grant 62488101 and in part by the Peng Cheng Laboratory and China Mobile (PCLCMCC) Foundation for Science and Innovation under Grant 2024ZY2B0070. *(Siqi He and Haozhe Zhu contributed equally to this work.) (Corresponding authors: Haozhe Zhu; Chixiao Chen.)*

Siqi He, Haozhe Zhu, Hongyi Zhang, Yujie Ma, Zexing Chen, Mengjie Li, Danfeng Zhai, Qi Liu, Xiaoyang Zeng, and Ming Liu are with the State Key Laboratory of Integrated Chips and Systems, College of Integrated Circuits and Micro-Nano Electronics, Fudan University, Shanghai 200433, China (e-mail: zhuhz@fudan.edu.cn).

Chixiao Chen is with the State Key Laboratory of Integrated Chips and Systems, College of Integrated Circuits and Micro-Nano Electronics, Fudan University, Shanghai 200433, China, and also with Shaoxing Laboratory, Shaoxing, Zhejiang 312000, China (e-mail: cxchen@fudan.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/JSSC.2025.3571315.

Digital Object Identifier 10.1109/JSSC.2025.3571315

I. Introduction

LARGE language models (LLMs) have demonstrated remarkable performance in complex artificial intelligence (AI) tasks, particularly in natural language processing and multi-tasking [1], [2], [3], [4]. Unlike conventional Transformer-based models [5], [6], [7], which typically comprise millions to billions of parameters, LLMs scale to tens or even hundreds of billions. This massive scaling enables significant improvements in generalization capabilities and long-context understanding. However, it also intensifies the communication overhead between processors and memory, which severely impacts memory bandwidth and energy efficiency (EEF). As a result, deploying LLMs in resource-constrained environments poses significant challenges [8], [9], [10].

Recently, compute-in-memory (CIM) architectures have emerged as a promising solution for edge acceleration of deep neural networks (DNNs). By integrating computational logic directly into memory cells, CIM architectures fundamentally minimize data movement between processors and memory, thereby addressing critical on-chip memory bandwidth limitations. To accommodate diverse computational demands, various integer (INT) and floating-point (FP) CIM designs have been developed, demonstrating significant improvements in EEF and parallelism for both INT multiply-and-accumulate (MAC) and FP-MAC operations [11], [12], [13], [14], [15], [16], [17], [18]. These advancements position CIM architectures as a robust solution for accelerating both DNN inference and training tasks.

In the context of LLM inference, it is crucial to assess whether CIM architectures can still deliver system-level performance improvements. Given that LLM inference comprises

0018-9200 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.

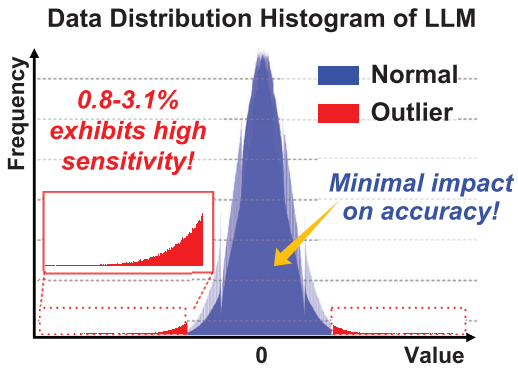


Fig. 1. Weight distribution of the OPT-1.3B model. The outliers depicted in the figure are for illustrative purposes only, highlighting representative examples that could lead to significant losses in inference accuracy even with slight variations. The range of 0.8%–3.1% represents the final outlier ratio achieved by our quantization algorithm.

two main phases (prefilling and decoding), the effectiveness of CIM depends on its data reuse characteristics. The prefilling phase inherently achieves high data reuse through the parallel processing of input tokens, while the decoding phase sustains data reuse by employing batching techniques to group multiple inference requests. By significantly reducing on-chip data movement associated with these reused data, CIM architectures effectively address the on-chip memory bandwidth bottleneck. Consequently, CIM continues to offer a compelling solution for LLM deployment.

Despite their potential, current CIM architectures are hindered by insufficient memory capacity when applied to LLMs. This limitation forces frequent weight retrieval from DRAM, which severely impacts EEF and significantly increases inference latency [19]. Although low-bit quantization techniques have been proposed to compress DNN models and reduce data movement [20], they are less effective for LLMs and often demand significant retraining efforts. Recent research indicates that performance degradation in quantized LLMs is largely caused by a small fraction of outliers, especially when the model size exceeds a certain threshold (see Fig. 1) [21], [22], [23]. To mitigate this, outlier-aware quantization (OAQ) techniques have been introduced, preserving the FP format for outliers while quantizing normal values to low-precision INTs [21], [22], [23], [24], [25], [26]. Among these, [21] is the first to implement a hybrid encoding of outlier and normal values, striking a balance between model accuracy and memory footprint. However, implementing outlier-aware (OA) LLM inference on CIM architectures still faces several critical challenges.

The first challenge is achieving OA computation with high EEF. As shown in Fig. 2 ***Challenge*1.(a)**, current mixed-precision CIM architectures are not inherently optimized for OA operation. Some designs rely on a single CIM macro that switches between fully INT or FP modes and thus cannot handle element-wise, randomly distributed outliers [27], [28], [29]. Others employ separate INT and FP cores to perform mixed-precision computation but suffer from high hardware overhead and unbalanced workloads [30], [31]. This highlights the need for a truly OA-CIM architecture that strikes a balance between memory footprint, hardware efficiency, and inference accuracy. Furthermore, as illustrated in ***Challenge*1.(b)**, traditional low-power MAC strategies are insufficient for OA-CIM. While previous LUT-based approaches [28], [32] reduce the number of adders by precomputing sums of adjacent weights and reusing them across different activation bits, they fail to address the complexities of hybrid outlier-normal value computation. This is because, in such scenarios, FP outliers require mantissa alignment before being summed with INT normal values, leading to increased operand bit-width. This bit-width expansion directly escalates the number of required adders, resulting in substantial hardware overhead that renders LUT-based methods impractical. This limitation highlights the necessity for specialized MAC solutions in OA-CIM. Additionally, as shown in ***Challenge*1.(c)** of Fig. 2, integrating FP-MAC operations into the CIM macro introduces the need for additional circuits to handle exponent addition, comparison, and subtraction, which are essential for mantissa alignment. However, existing exponent processing schemes in FP dataflow often suffer from long execution cycles due to time-serial exponent comparisons [33], [34] or incur significant area and power overheads due to complex alignment circuits [35]. Therefore, designing an energy-efficient MAC scheme compatible with both FP and INT modes, along with optimized exponent processing is essential.

The second challenge arises from the power-intensive readout circuit in row-wise CIM architectures, as shown in ***Challenge*2** of Fig. 2. During the decoding stage, LLM algorithms typically require extensive weight parameters for matrix-vector multiplication, exemplified by the generation of query, key, and value activations ($Q/K/V$ generation) within attention blocks and the linear layers of feed-forward networks (FFN). Even when batching and scheduling techniques are employed to group multiple matrix-vector operations into matrix-matrix multiplications, LLM inference still exhibits a lower weight reuse rate compared to convolutional and matrix-matrix multiplication operations in DNNs. Therefore, row-wise CIM [36] offers a memory-to-compute ratio better aligned with the demands of LLMs than crossbar architectures. However, it incurs frequent bitline charging and discharging, resulting in high-power consumption.

To address the aforementioned challenges, a 22-nm OA-CIM macro is designed and fabricated to support LLM inference. The key contributions are as follows.

1) An algorithm-hardware co-designed OA-CIM architecture is proposed to facilitate element-wise hybrid processing of BF16 outliers and INT4 normals, ensuring memory alignment while optimizing memory usage and accuracy.
2) A FP/INT-compatible LUT (FIC-LUT)-based MAC circuit is introduced, with an XOR-sharing exponent processing (XOR-SEP) scheme implemented to gate non-maximum exponent comparisons for improved area efficiency and latency.
3) A distribution-offset weight encoding (DOWE) scheme, coupled with a sparsity-aware readout circuit, is

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.

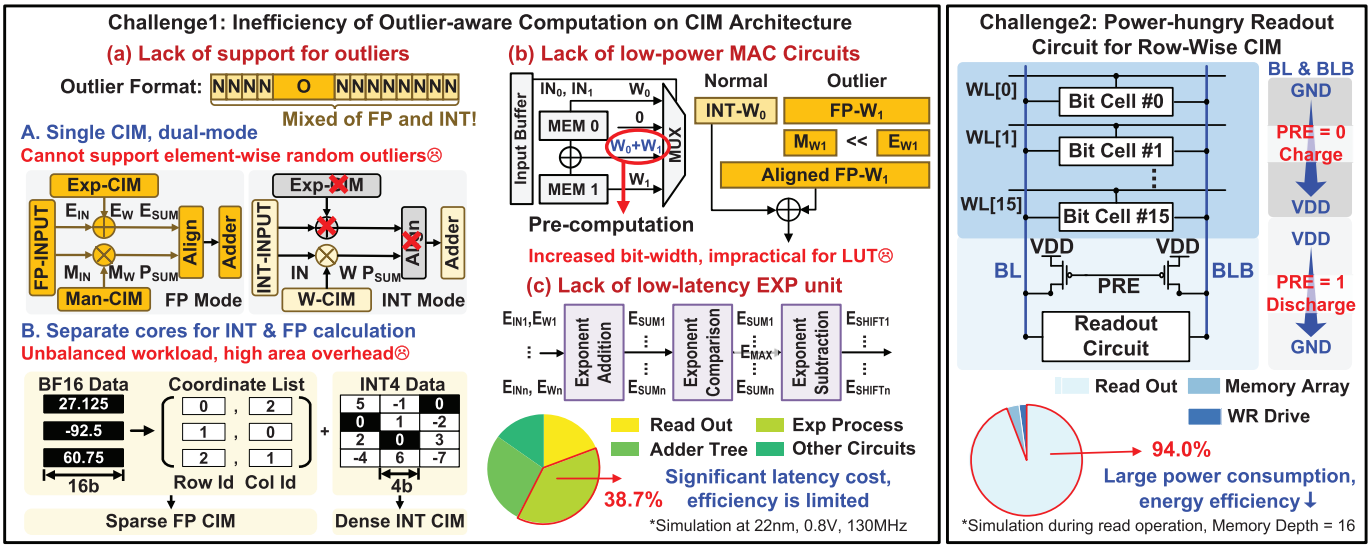


Fig. 2. Key challenges in designing a CIM processor for LLM acceleration: Inefficient handling of outliers and high-power readout operations. (a) Lack of support for outliers. (b) Lack of low-power MAC Circuits. (c) Lack of a low-latency EXP unit.

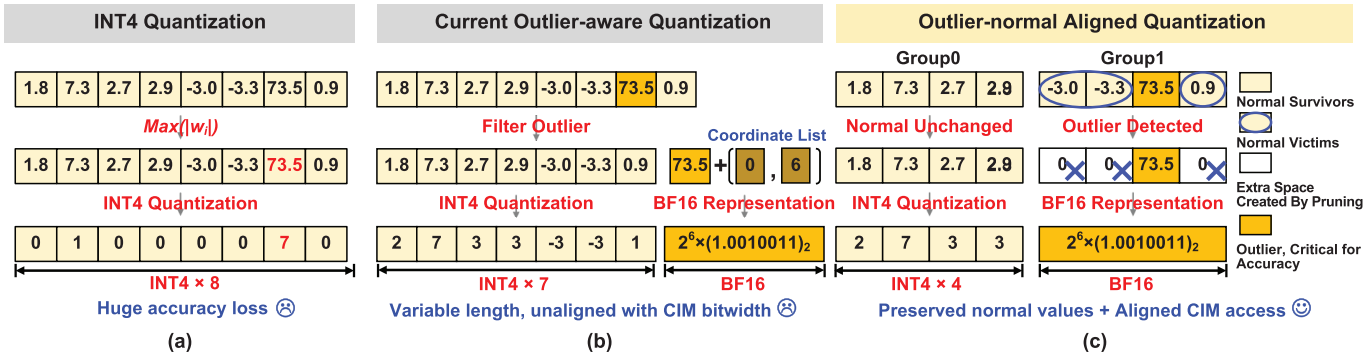


Fig. 3. Quantization methods comparison. (a) INT4 quantization. (b) Current OAQ. (c) Outlier-normal aligned quantization.

proposed to leverage enhanced bit-wise sparsity for higher readout efficiency.

4) A 22 nm prototype of the proposed OA-CIM macro is fabricated, achieving an EEF of 346.6 TOPS/W in the INT4 mode and 249.5 TFLOPS/W in the outlier mode.

The article is organized as follows. Section II presents the co-designed OA-CIM architecture. Section III details the design for energy-efficient OA computation, highlighting the FIC-LUT MAC circuit and the XOR-SEP scheme. Section IV introduces the DOWE technique. Section V provides the measurement and comparison results. Finally, Section VI concludes this article.

II. OA CIM Architecture

This section introduces the OA-CIM architecture co-design, highlighting the outlier-normal aligned quantization (ONAQ) algorithm and the OA-CIM macro implementation. We open-source the ONAQ algorithm on GitHub.[1]

[1]The code is available at https://github.com/zhutmost/llm-outlier-quan

A. Outlier-Normal Aligned Quantization

To reduce the memory footprint and computational overhead in LLM inference, various OAQ algorithms have been developed. These algorithms assess the sensitivity of each weight element, isolating outliers that risk significant accuracy loss even with minor variations. By retaining these outliers in high precision, OAQ achieves better accuracy than low-bit quantization. A classic approach is to preserve outliers in BF16 for its wide dynamic range, while quantizing normal values to INT4, which aligns well with modern memory architectures. This strategy effectively balances precision for critical outliers with efficiency for the majority of the data. However, as shown in Fig. 3(b), existing OAQ frameworks often introduce extra coordinate indices or bitmasks along with complex decoding logic to locate outliers, along with variable-length storage format that hinders the EEF of current CIM architectures.

To resolve this limitation, this work proposes ONAQ, an OAQ framework inspired by [21]. The foundational observation from [21] is that outlier values play a critical role in maintaining model accuracy, while the normal values adjacent to them have relatively lower significance. Leveraging this insight, ONAQ strategically sacrifices these low-impact normal

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.

TABLE I
INFERENCE PERFORMANCE OF ONAQ COMPARED WITH OTHER QUANTIZATION METHODS

Method	OPT-1.3B			OPT-2.7B			OPT-6.7B			OPT-13B		
	Wiki	PTB	C4	Wiki	PTB	C4	Wiki	PTB	C4	Wiki	PTB	C4
BF16	14.62	16.96	14.72	12.47	15.11	13.16	10.86	13.09	11.74	10.13	12.34	11.20
INT4	15.09	17.63	15.29	13.43	16.62	14.19	11.49	14.45	12.6	10.55	12.84	11.55
ONAQ	14.71	17.37	14.97	12.42	15.40	13.35	10.90	13.22	11.83	10.19	12.43	11.27

*Lower perplexity correspond to better performance.

values and reallocates its storage space to provide a wider bit-width for outlier representation. As illustrated in Fig. 3(c), when an outlier like *73.5* is detected in group *2*, the remaining three elements with lower contributions to MAC results are masked to zero. This allows the BF16 outlier *73.5* to occupy the freed space in the original matrix. This approach not only improves storage efficiency but also significantly expands the representable range of outliers, thereby enhancing the overall model accuracy.

We implement the ONAQ algorithm using the GPTQ iterative method [37], a popular approach in LLM quantization for its high parallelism and efficient memory access. In each iteration, outliers are selected based on their sensitivity. To define the sensitivity of an element, we first establish a baseline quantization error $\varepsilon_{W_{\text{INT4}}}$ by quantizing the entire weight matrix W to INT4 format and computing the L2-norm difference between $W_{\text{INT4}}X$ and WX, where X is a small set of calibration inputs. For each element $w_{i,j}$ in the matrix, we preserve it in BF16 format, zero out its adjacent three elements, and quantize the remaining weights to INT4, generating a modified weight matrix $W_{i,j}$. The sensitivity of $w_{i,j}$ is then assessed as the difference between $(\varepsilon_{W_{\text{INT4}}})^2$ and $(\varepsilon_{W_{i,j}})^2$, where $\varepsilon_{W_{i,j}}$ is the quantization error of $W_{i,j}$. An element is classified as an outlier if its sensitivity exceeds a threshold $s_{\text{th}} = r \times \tau$, where τ is the Hessian-weighted mean value of the channel variance and r is a designer-adjustable parameter to control the outlier ratio.

To further reduce hardware overhead and minimize the memory footprint of outlier indices, in addition to setting a sensitivity threshold for outlier detection, we imposed constraints on the outlier distribution. Specifically, we enforced a rule that within a block of b elements, there can be at most a outliers (in this study, $b = 32$ and $a = 1$). For outliers exceeding this proportion, we rigorously discard them. This mechanism ensures that, even when multiple elements within a group of four consecutive values exhibit high quantization loss sensitivity, the number of outliers remains controlled, thereby maintaining the robustness and efficiency of the ONAQ data format.

We conducted a comprehensive comparative analysis of BF16 baseline, INT4 quantization, and ONAQ across multiple scales of the OPT model. As shown in Table I, on the Wikitext-2 [38], PTB [39], and C4 [40] datasets, ONAQ demonstrates accuracy closely matching that of the BF16 baseline, with only a 0.9% average increase in perplexity. In contrast, INT4-only quantization exhibits an average accuracy loss of 6.0% relative to the baseline. Thus, ONAQ achieves a 5.1% improvement in accuracy compared to INT4-only quantization while maintaining comparable memory efficiency.

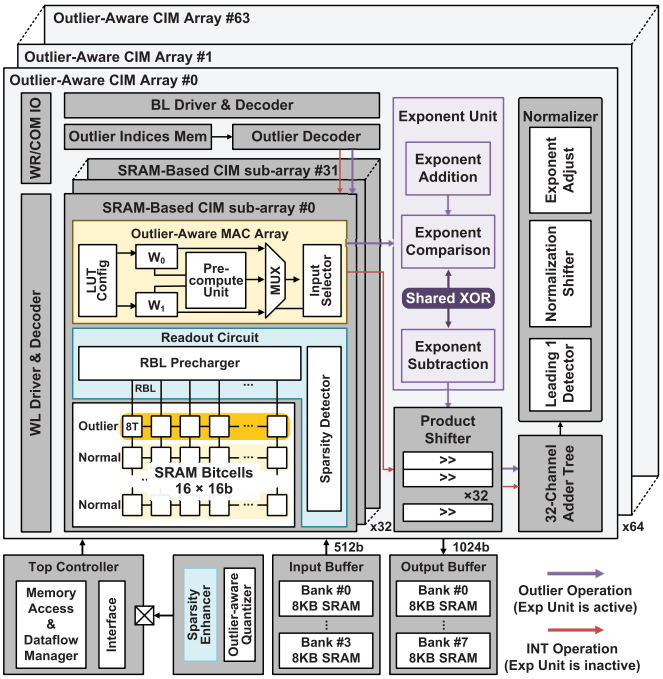


Fig. 4. Overall architecture of OA-CIM macro.

The proposed ONAQ framework primarily focuses on weight quantization for LLMs, preserving outliers in BF16 while compressing normal weights to INT4. For activation quantization, our current design employs a uniform INT4 scheme, which has demonstrated excellent performance on the OPT model family. Given that the effectiveness of the ONAQ framework is influenced by the weight and activation distributions, its generalization capability on more complex model architectures may require further validation.

B. OA CIM Macro

Fig. 4 illustrates the overall architecture of the OA-CIM macro, which aligns with the ONAQ algorithm. The OA-CIM macro consists of 64 512 × 16b CIM arrays. Each CIM array consists of the write/compute interfaces with word/bitline drivers, 32 CIM sub-arrays paired with 32 MAC arrays, an outlier decoder with indices memory, an exponent processing unit, a product shifter, a channel-wise adder tree, and a normalization unit. Each CIM sub-array has 16 × 16b CIM cells, with every 16b representing either 4× INT4 normal values or 1× BF16 outlier. During computation, weights and outlier indices are fetched from the CIM array and the indices memory, respectively. Based on the indices, the outlier decoder configures the operation mode of the MAC circuit and the exponent unit to obtain MAC values (MACV) and exponent differences separately. The product shifter then aligns the MACV according to the exponent differences and feeds the result to the channel-wise adder tree, followed by the normalization unit. Notably, the entire computation process is executed within one cycle through a fully parallel implementation. The critical path, starting from weight retrieval in the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.

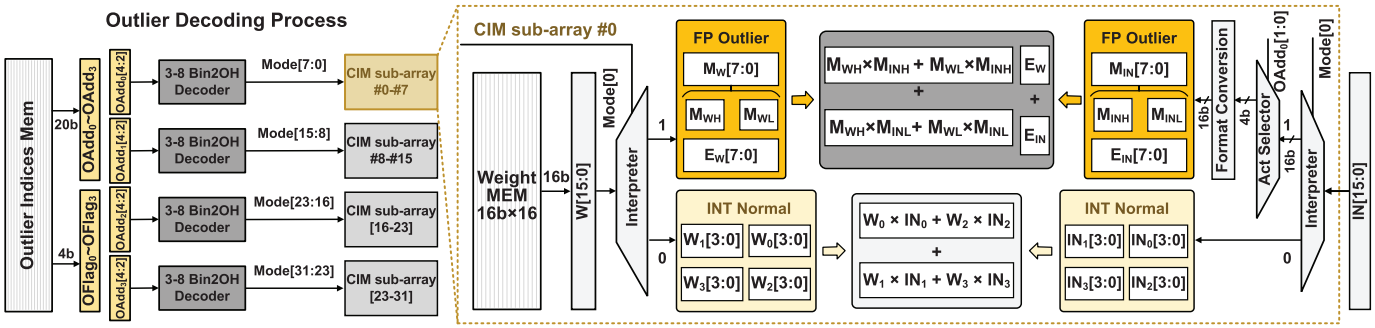


Fig. 5. Outlier-normal decoding process and mode configuration.

CIM macro and ending with final result quantization in the normalization unit, is implemented as purely combinational logic without any pipeline registers. This design ensures seamless operation for both the exponent and mantissa processing and avoids pipeline stalls, therefore maximizing computational efficiency and throughput.

Fig. 5 illustrates the decoding process and mode configuration for hybrid outlier-normal value computation in detail. Before computation begins, the quantized weights are stored in the weight memory, while the outlier indices, determined based on the outlier selection results of the ONAQ algorithm, are stored in the outlier indices memory. As previously mentioned, the ONAQ process limits the outlier ratio to a maximum of 1/32, meaning each 32-value block requires only a 6-bit outlier index. This index consists of two parts: 5 bits for the outlier address (OAdd, indicating the position of the outlier within the block) and 1 bit as an outlier flag (OFlag, indicating whether any outliers exist in the block). For the proposed OA-CIM array, which has 32 CIM sub-arrays (corresponding to four 32-value blocks), four 6-bit outlier indices (OAdd_0-OAdd_3 and OFlag_0-OFlag_1) are required to locate outliers, as shown in Fig. 5.

To better understand the mode configuration, let us examine CIM sub-array #0 as an example. The higher bits of the outlier address, $\text{OAdd}_0[4:2]$, are processed by binary-to-one-hot decoders to generate the mode signal, Mode[7:0], which corresponds to CIM sub-arrays #7-0. Specifically, Mode[0] is sent to sub-array #0. When OFlag_0 or Mode[0] is 0, CIM sub-array #0 operates in the INT4 mode and reads 16 bits, interpreted as four INT4 weights. Together with four INT4 activations, the four INT4 weights are sent to the MAC array to perform $\text{MACV} = \sum_{i=0}^{3} W_i \times \text{IN}_i$. Otherwise, when OFlag_0 is 1 and Mode[0] is 1, CIM sub-array #0 operates in FP-outlier mode, and the 16-bit weight is treated as a BF16 number. Meanwhile, IN[15:0], which corresponds to four INT4 activations, is processed by the activation selector to select the INT4 activation corresponding to the weight based on the lower bits of the outlier address, OAdd[1:0]. This selected INT4 activation is then converted into a BF16 value using a data format conversion module, ensuring it matches the original INT4 value. Finally, the resulting BF16 activation is multiplied by the BF16 outlier weight to complete the computation.

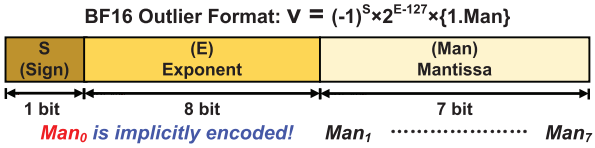


Fig. 6. Data format of BF16-outlier, highlighting the implicitly encoded leading one in the mantissa.

As shown in Fig. 6, the BF16 outlier value consists of a 1-bit sign, an 8-bit exponent, and a 7-bit mantissa. In this work, we adhere to the IEEE-754 standard [41], which implies an implicitly encoded leading "1" in the mantissa. Thus, the BF16-outlier value can be represented as $v = (-1)^S \times 2^{E-127} \times (1.\text{Man})$. To align with the application requirements of this study and maintain consistency with previous FP-CIM architectures, we exclude support for subnormal BF16 numbers. Nevertheless, the proposed framework can be extended to fully IEEE-compliant modes if necessary. In the following, we denote the combined value of the hidden "1" and mantissa of weight as $M_W[7:0] = \{1, \text{Man}_W[7:0]\}$, which is further divided into $M_{\text{WH}}[3:0]$ and $M_{\text{WL}}[3:0]$. Similarly, for the activation, the combined value is denoted as $M_{\text{IN}}[7:0] = \{1, \text{Man}_{\text{IN}}[7:0]\}$, which is then split into $M_{\text{INH}}[3:0]$ and $M_{\text{INL}}[3:0]$. Both $M_W[7:0]$ and $M_{\text{IN}}[7:0]$ are sent to the MAC array to perform multiplication.

The above outlier decoding scheme achieves a 3.82× reduction in memory usage while taking up only 0.4% and 0.2% of the total area and power.

III. OA Computing Circuit

This section details the implementation for efficient OA computation, featuring the FIC-LUT circuit to minimize MAC overhead and the XOR-SEP to optimize exponent processing.

A. FIC LUT-Based MAC Circuit

Previous LUT-based techniques [28], [32] sought to minimize the number of full adders by precomputing all possible MAC results for a group of weights with 1-bit activations. The appropriate results were then selected based on the actual activation values. However, in FP-MAC operations, directly

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.

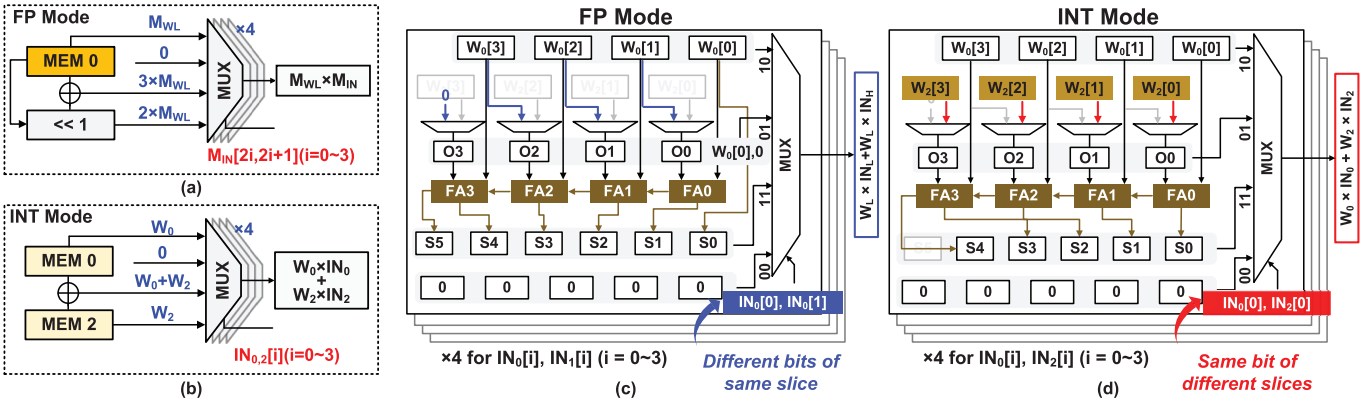


Fig. 7. FIC-LUT reconfigured as (a) FP-MAC and (b) INT-MAC operations. Detailed configurations of the FIC-LUT in (c) FP and (d) INT modes. With $M_W = \{M_{\mathrm{WH}}, M_{\mathrm{WL}}\} = \{W_1, W_0\}$ and $M_{\mathrm{IN}} = \{\mathrm{IN}_1, \mathrm{IN}_0\}$, the figure demonstrates FP-MAC operations using an INT format, highlighting the flexibility of the reconfigurable approach.

accumulating mantissas is incorrect because adjacent weights may have differing exponents, as illustrated as follows:

$$\begin{aligned}&\mathrm{IN}_0 \times 2^{E_{W_0}} \times M_{W0} + \mathrm{IN}_1 \times 2^{E_{W_1}} \times M_{W1} \\ &\neq 2^{E_{W_0}} \times (\mathrm{IN}_0 \times M_{W0} + \mathrm{IN}_1 \times M_{W1}). \end{aligned} \tag{1}$$

To facilitate weight reuse in FP-MAC operations, OA-CIM incorporates the FIC-LUT MAC circuits. As detailed in Section II-B, when Mode[i] is set to 1, the MAC array computes MACV = $M_W[7{:}0] \times M_{\mathrm{IN}}[7{:}0]$, which can be further decomposed as follows:

$$\begin{aligned}&M_W \times M_{\mathrm{IN}} \\ &= \sum_{i=0}^{3} (M_{\mathrm{IN}}[2i] \times (M_{\mathrm{WH}} \ll 4 + M_{\mathrm{WL}}) \\ &\quad + M_{\mathrm{IN}}[2i+1] \times (M_{\mathrm{WH}} \ll 4 + M_{\mathrm{WL}}) \times 2) \ll 2i. \end{aligned} \tag{2}$$

This decomposition reveals opportunities for weight reuse in FP-MAC operation. For example, by grouping terms such as $M_{\mathrm{IN}}[2i] \times M_{\mathrm{WL}}$ and $M_{\mathrm{IN}}[2i+1] \times M_{\mathrm{WL}} \times 2$, the computation can be reformulated as a selection operation among M_{WL}, $2 \times M_{\mathrm{WL}}$, $3 \times M_{\mathrm{WL}}$ or 0, depending on the values of $M_{\mathrm{IN}}[2i]$ and $M_{\mathrm{IN}}[2i+1]$. Fig. 7(a) (top) shows the corresponding LUT entry configuration. The first LUT entry stores the original weight mantissa M_{WL} and is accessed when $M_{\mathrm{IN}}[2i+1], M_{\mathrm{IN}}[2i] = 01$. The second entry, $2 \times M_{\mathrm{WL}}$, is generated through a shift operation and is selected when $M_{\mathrm{IN}}[2i+1], M_{\mathrm{IN}}[2i] = 10$. Fig. 7(a) (bottom) depicts the activated blue path through the multiplexer, allowing the precomputation (PC) adders to produce the third entry, $3 \times M_{\mathrm{WL}}$ for $M_{\mathrm{IN}}[2i+1], M_{\mathrm{IN}}[2i] = 11$. The fourth entry, set to 0, is addressed when $M_{\mathrm{IN}}[2i+1], M_{\mathrm{IN}}[2i] = 00$. These four entries are then forwarded to four result multiplexers, each using two adjacent bits of $M_{\mathrm{IN}}[7:0]$ as the selection signal, ultimately yielding $M_{\mathrm{IN}} \times M_{\mathrm{WL}}$. The remaining component of MACV, $M_{\mathrm{IN}} \times M_{\mathrm{WH}}$, is calculated in a similar way.

Conversely, if Mode[i] is set to 0, the MAC array calculates MACV = $\sum_{i=0}^{3} W_i \times \mathrm{IN}_i$, which can be written as

$$\begin{aligned}\sum_{i=0}^{3} W_i \times \mathrm{IN}_i = \sum_{i=0}^{3} (&\mathrm{IN}_0[i] \times W_0 + \mathrm{IN}_2[i] \times W_2 \\ &+ \mathrm{IN}_1[i] \times W_1 + \mathrm{IN}_3[i] \times W_3) \ll i. \end{aligned} \tag{3}$$

As noted in prior work, the above computation also provides opportunities for weight reuse. For instance, by grouping the terms $\mathrm{IN}_0[i] \times W_0$ and $\mathrm{IN}_2[i] \times W_2$, the calculation can also be expressed as a selection among W_0, W_2, $W_0 + W_2$, or 0, depending on the values of $\mathrm{IN}_0[i]$ and $\mathrm{IN}_2[i]$. Fig. 7(b) (top) illustrates the corresponding LUT entry configuration. The first and second LUT entries store the initial weights, W_0 and W_2, which are accessed when $\mathrm{IN}_0[i], \mathrm{IN}_2[i]$ are 10 and 01, respectively. Fig. 7(b) (bottom) shows the activated red path through the multiplexer, which enables the PC adders to produce the third entry, $W_0 + W_2$, for $\mathrm{IN}_0[i], \mathrm{IN}_2[i] = 11$. The fourth entry, set to 0, is accessed when $\mathrm{IN}_0[i], \mathrm{IN}_2[i] = 00$. These entries are then directed to four result multiplexers, each using one bit of $\mathrm{IN}_0[i]$ and $\mathrm{IN}_2[i]$ as the selection signal, ultimately yielding $\mathrm{IN}_0 \times W_0 + \mathrm{IN}_2 \times W_2$. The remaining component of MACV, $W_1 \times \mathrm{IN}_1 + W_3 \times \mathrm{IN}_3$, is computed similarly.

To demonstrate the efficiency of FIC-LUT, we evaluated the normalized energy consumption for MAC operations in our approach against other leading methods at 0.8 V, 130 MHz. The results show that FIC-LUT achieves energy reductions of 4.76×, 2.64×, 2.45×, and 1.93× compared to the baseline without LUT integration, as well as Radix4 [42], Radix16 [43], and Radix16+ LUT scheme [43], respectively. These savings stem from a more optimal balance between data reuse frequency and the number of PC adders, along with reduced overhead in control signal generation. Additionally, the overall CIM macro area is reduced by 7% compared to the baseline.

B. XOR-Sharing Exponent Processing Scheme

In this work, we employ a product-alignment strategy for FP-MAC operation, which utilizes the maximum product exponent, defined as the summation of $\mathrm{E_{IN}}$ and E_W, as the alignment reference for the products generated by FIC-LUT. This alignment scheme enables further accumulation with the least data truncation, thereby enhancing inference accuracy. Fig. 8 depicts the exponent computation flow within the OA-CIM macro. Initially, 32 pairs of $\mathrm{E_{IN}}$ and E_W are directed to

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.

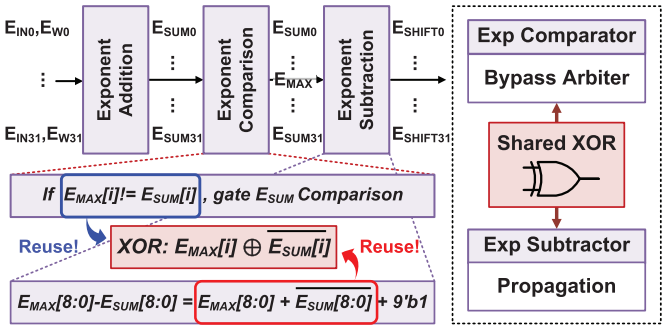


Fig. 8. Exponent processing flow, highlighting the shared XOR logic for exponent comparison and subtraction.

the addition unit to compute the exponent summation, E_{SUM}. Subsequently, all E_{SUM} values are sent to the comparison unit to identify the maximum term, E_{MAX}. The remaining E_{SUM} values, along with the E_{MAX}, are then processed in the subtraction module to generate 32 sets of shift alignments, E_{SHIFT}.

By examining the logic employed to derive E_{SHIFT} and E_{MAX}, we have identified a shareable component, as illustrated in Fig. 8. Intuitively, a candidate E_{SUM} containing a bit $E_{SUM}[i]$ that differs from $E_{MAX}[i]$ cannot represent the maximum exponent. This comparison can be efficiently performed using a 1-bit XOR operation, expressed as $E_{MAX}[i] \oplus E_{SUM}[i]$. On the other hand, E_{SHIFT} is formulated as: $E_{MAX}[8:0]$ - $E_{SUM}[8:0]$ = $E_{MAX}[8:0]$ + $E_{SUM}[8:0]$ +1, in which the addition operation is carried out using the same XOR operation. Based on this insight, we propose sharing XOR gates between the exponent subtractor and comparator, where the XOR gates serve dual purposes: they function as the bypass arbiter in the comparator, facilitating the efficient identification of maximum exponents, and act as propagation signals in the full adder, enabling subtraction operations.

Fig. 9(a) presents the gate-level design of the bit-serial exponent comparator, utilizing shared XOR gates as a bypass arbiter. The comparator determines the maximum E_{SUM_n} via a three-phase process. In phase-1, the most significant bits of each E_{SUM_n} are fed to a NOR gate to generate $\overline{E_{MAX}[8]}$. Phase-2 employs XOR gates to compare $E_{SUM_n}[8]$ (n = 0-31) with $E_{MAX}[8]$, generating $Bypass_n[7]$ (n = 0-31) for the next phase. Phase-3 uses AND gates to gate further comparisons based on the XOR results. For instance, in Fig. 9(a), both $E_{SUM_1}[8]$ and $E_{SUM_{31}}[8]$ are logic'1, so $\overline{E_{MAX}[8]} = 0$. In phase-2, the XOR gate detects that $E_{SUM_0}[8] \neq E_{MAX}[8]$, thus enabling $Bypass_0[7]$. Consequently, each bit of $E_{SUM_0}[7:0]$ is gated to logic'0 before participating in the subsequent NOR-based E_{MAX} calculation. This three-phase process is repeated for each bit of all E_{SUM} values, ultimately yielding E_{MAX}. Notably, all three phases are implemented within a single cycle, and no pipeline registers are introduced between them. This design avoids pipeline bubbles, thereby maintaining high throughput and computational efficiency.

Fig. 9(b) provides a simple example illustrating the non-maximum exponent gating process. At each step, $E_{MAX}[i]$ (i = 8-0) is determined using a NOR gate. If $E_{MAX}[i]$ =1, all remaining bits of E_{SUM_n} with a 0 in the ith bit position are gated to zero, thus gating subsequent comparisons. This occurs in step ❶ for $E_{SUM_0}[7:0]$ and in step ❷ for $E_{SUM_2}[6:0]$. Conversely, when $E_{MAX}[i]$ =0, no gating process is triggered as no non-maximum E_{SUM_n} can be identified. This is demonstrated in step ❸.

Fig. 10 illustrates the circuit-level design of the exponent subtractor, which employs a carry bypass adder architecture based on the standard propagation (P) and generation (G) signals. Here, P is defined as $E_{MAX}[i] \oplus E_{SUM}[i]$, implemented using the shared XOR gates. By cascading full adders constructed with P and G signals, the computation of $E_{MAX}[8:0]$ + $E_{SUM}[8:0]$ is achieved, while the remaining increment operation for E_{SHIFT} calculation is replaced by an equivalent product shifting process.

The proposed bit-serial exponent comparator provides significant area and power savings over traditional schemes by eliminating the need for a large comparator tree without incurring the latency penalty of time-serial exponent alignment. The use of shared XOR gates and a non-maximum exponent gating strategy cuts down redundant logic and unnecessary operands, leading to further area and energy improvements. Simulated at 0.8 V and 130 MHz, the XOR-SEP scheme achieves a 1.54× reduction in latency and a 1.43× decrease in area for the exponent processing unit compared to the non-XOR-sharing baseline [44]. To further evaluate the scalability of the XOR-SEP scheme, we conducted a comprehensive evaluation across various exponent bit-widths, including FP16 (5-bit exponent), FP32 and BF16 (8-bit exponent), and FP64 (11-bit exponent). Evaluated at 0.8 V and 130 MHz, the XOR-SEP scheme achieves 1.72×, 1.54×, and 1.14× speedup for 5-bit, 8-bit, and 11-bit exponents, respectively, compared to the baseline. While the performance advantage decreases as the exponent bit-width increases, XOR-SEP consistently outperforms the baseline in accelerating the exponent processing process.

IV. Distribution-Offset Weight Encoding

This section focuses on optimizing the readout process of OA-CIM, utilizing a sparsity-aware readout circuit to exploit the bit-level sparsity induced by the DOWE scheme.

Fig. 11(a) presents the sparsity-aware readout circuit [45], while Fig. 11(b) details the readout power across different cases. For S = 1, the readout power at W = 0 is 11.5× lower than that at W = 1, a reduction resulting from the elimination of read bitline (RBL) discharge at W = 0. This motivates us to increase the bit-wise sparsity in LLM weights further to reduce energy consumption during readout.

Fig. 11(c) presents a schematic illustrating the weight distribution in an LLM layer, which, as demonstrated in previous studies and our experiments, exhibits a noticeable Gaussian pattern [46]. Under this distribution, many values concentrate around zero, with a significant portion being negative. In the 2's complement representation, negative values often contain consecutive logic'1, as seen in $-1(1111)_2$ and $-2(1110)_2$. This high density of logic'1 in near-zero negative values leads to increased readout power consumption. To mitigate this problem, we applied a per-channel shift to the quantized weights. Given a weight matrix $W \in \mathbb{R}^{C_O \times C_I}$ and an activation

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.

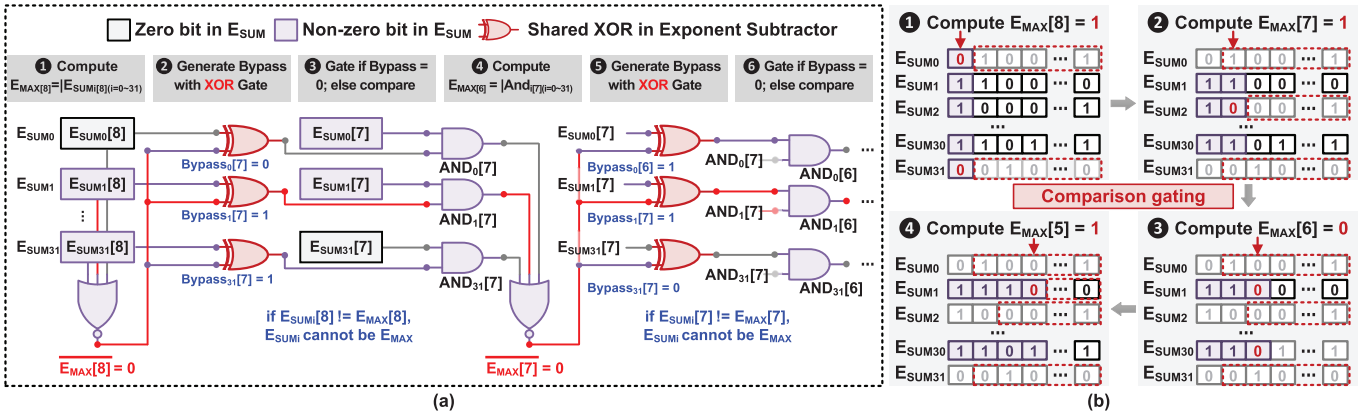


Fig. 9. (a) Construction of the bit-serial exponent comparator with the shared XOR logic. (b) Example of non-maximum exponent comparison gating.

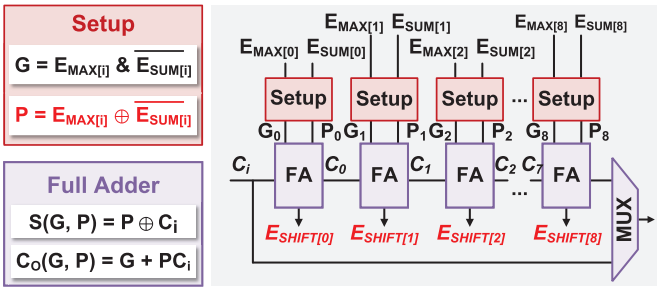


Fig. 10. Construction of the exponent subtractor with the shared XOR logic.

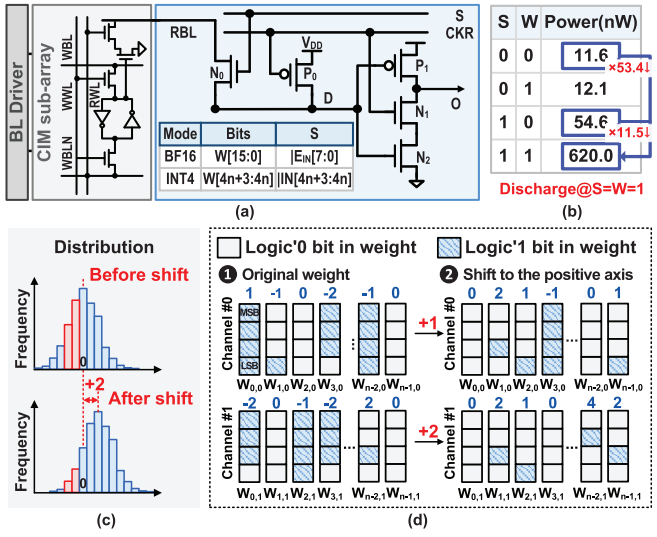


Fig. 11. (a) Sparsity-aware readout circuit. (b) Power consumption table for different readout cases, evaluated at 0.8 V, 25 °C, and 130 MHz, based on one CIM column with 16 cells. (c) Gaussian-like distribution of LLM weights. (d) Illustration of the DOWE scheme.

vector $A \in \mathbb{R}^{C_I \times 1}$, the output for the nth channel is expressed as

$$O_n = \sum_{m=0}^{C_I-1} \left(W_{n,m} + \text{Offset}(m)\right) \cdot A_m - \sum_{m=0}^{C_I-1} \text{Offset}(m) \cdot A_m. \quad (4)$$

Here, C_O and C_I denote the output and input channel counts, respectively. Offset(m) represents the offset selected for the

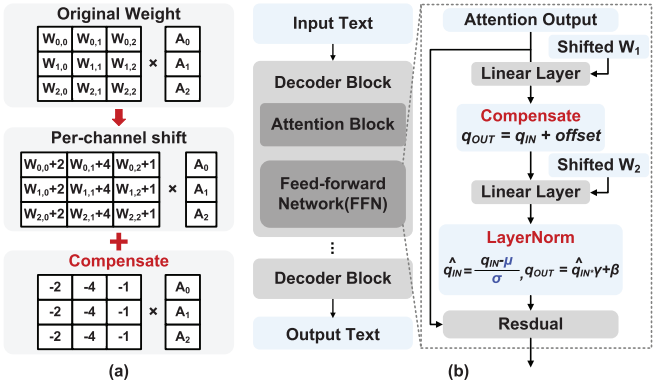


Fig. 12. (a) DOWE requires compensation to maintain MAC operation accuracy. (b) Illustration of how to compensate for the weight offset.

mth input channel, independent of the activation. Our method for determining the optimal offset involves an exhaustive search across all possible offset values. Based on empirical data, we evaluate offsets ranging from 0 to 4 for INT4 normal values. The offset that yields the highest bit-wise sparsity is selected as the final solution (i.e., the proportion of "0"s in the binary representation of the quantized values). Our experimental results show that the optimal offset is often 1 or 2.

As shown in Fig. 11(d), by selecting an optimal shift for each input channel offline, the weights are shifted rightward, lowering the proportion of near-zero negative values and substantially improving bit-level sparsity.

A critical issue with DOWE is the potential overflow of large positive values. To ensure that the DOWE scheme does not suffer from precision loss due to overflow, we integrate it with our proposed ONAQ framework. Specifically, if adding an offset causes a normal value to overflow, it is converted into an outlier and preserved in full BF16 precision. This approach avoids saturating large positive values and maintains inference accuracy.

On the other hand, to ensure correctness in matrix-vector multiplication with shifted weights in LLMs, a compensation step is theoretically required, where each feature in the output vector is adjusted by a uniform offset, as shown in Fig. 12(a). However, given the LLM computation flow, the final linear

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.

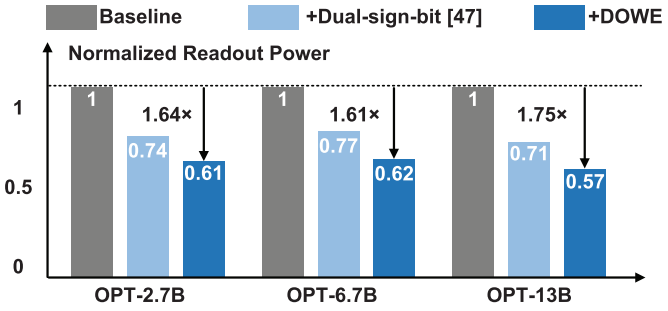


Fig. 13. Evaluation for the readout power under different weight-sparse encoding schemes on multiple OPT family models. Baseline: pre-trained model under ONAQ. "+Dual-sign-bit": optimize the baseline with dual-sign-bit encoding scheme in [47]. "+DOWE": optimize the baseline with DOWE.

layers of the attention and FFN blocks do not require compensation, as layer normalization subsequently normalizes the values using the mean and standard deviation, expressed as $\hat{q_{\text{IN}}} = (q_{\text{IN}} - \mu)/\sigma$. With the uniform offset, this transformation becomes $\widehat{q_{\text{IN}}} = (q_{\text{IN}} + \text{offset} - \mu - \text{offset})/\sigma$, indicating that the weight encoding does not impact the final result. For the remaining linear layers in FFN blocks and $Q/K/V$ generation in attention blocks, incoming activations are scaled by the respective distribution-offset values, accumulated as the final offset, and compensated to the MAC results, represented as $q_{\text{OUT}} = q_{\text{IN}} + \text{offset}$. This process is implemented in the residual module, which performs vector-wise addition. Since input activations are reused by weights across all output channels, the compensation process imposes negligible overhead on inference latency.

To further reduce power consumption on the RBL, we employ a dynamic-range-driven pre-charge gating scheme for the input activations. As illustrated in Fig. 11(a), when $E_{\text{IN}} = 0$ in the BF16 mode or IN = 0 in the INT4 mode, the control signal S on transistor N_0 in the readout circuit is set to zero, thus skipping the RBL pre-charging process.

As illustrated in Fig. 13, experiments were conducted on multiple models from the OPT family. Evaluated at 0.8 V, 130 MHz, the DOWE scheme achieves a reduction in readout power consumption by 1.67× and 1.23× compared to the baseline and dual-sign-bit weight-sparse encoding [47], respectively. Furthermore, the dynamic-range-driven gating scheme contributes to an additional 1.18× overall power reduction. The proposed DOWE scheme offers two key advantages. First, its lossless encoding, enabled by a compensation step, ensures no degradation in inference accuracy. Second, DOWE is highly hardware-efficient. By maintaining data in 2's complement format both before and after encoding, it eliminates the need for complex decoding logic during the CIM readout process. Moreover, the DOWE method avoids the hardware overhead associated with sign-aware adder trees in post-readout multiplication operations, a requirement in the dual-sign-bit encoding scheme proposed in [47].

V. Implementation and Measurement Results

A. Chip Implementation and Evaluation Results

To validate the proposed OA-CIM macro, a prototype chip was fabricated using 22 nm CMOS technology. The chip

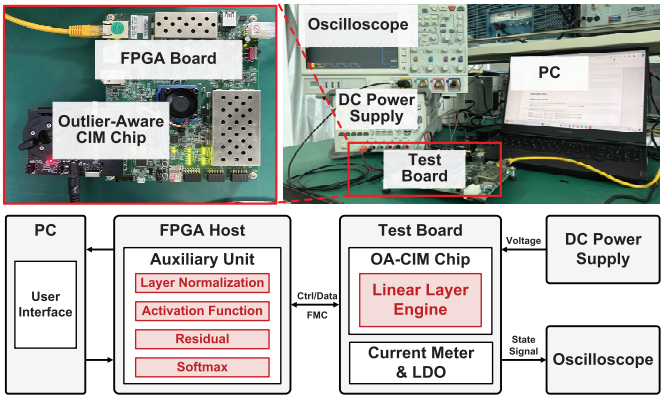
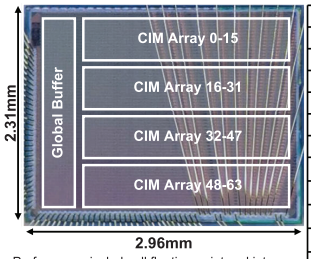


Fig. 14. Measurement setup of the OA-CIM chip.



Performance include all floating-point and integer operations: outlier decoding, in-memory exponent processing, mantissa shifting and normalization.
1. In outlier mode, the proportion of floating-point outliers ranges from 0.8% to 3.1%.
2. Measured at 50% input sparsity, highest peak energy efficiency point @VDD=0.60V, 55MHz (For fair comparison, bit-wise sparsity enhancement scheme is excluded).
3. Measured at 90% input sparsity, highest average energy efficiency point @VDD=0.60V, 55MHz (For fair comparison, bit-wise sparsity enhancement scheme is excluded).
4. Measured at 1.1V, 270MHz.

CHIP SUMMARY				
Technology		22nm CMOS		
Die area		2.96 × 2.31 = 6.84mm²		
Array area		33514.7um²		
Macro size		64 × 8Kb = 512Kb		
Supply voltage (V)		0.6 - 1.1		
Access time (ns)		3.7 - 18.1		
Mode		Outlier*1	INT4	BF16
Weight/Input precision (bit)		INT4+BF16	INT4	BF16
Output precision (bit)		BF16	BF16	BF16
Number of input channels		128	128	32
Number of output channels		64	64	64
Throughput (TOPS)		0.9-4.4	0.9-4.4	0.23-1.1
Average energy efficiency*2 (TOPS/W) (TFLOPS/W)		62.0-136.1	75.5-166.3	15.3-31.2
Peak energy efficiency*3 (TOPS/W) (TFLOPS/W)		109.3-249.5	167.2-346.6	37.1-74.9
Area efficiency*4 (TOPS/mm²) (TFLOPS/mm²)		2.06	2.06	0.52
Different LLM Models	OPT-6.7B @Wikitext-2	+0.04 (Perplexity↑)	+0.63 (Perplexity↑)	10.86 (Baseline)
	OPT-13B @Wikitext-2	+0.06 (Perplexity↑)	+0.42 (Perplexity↑)	10.13 (Baseline)

Fig. 15. Die photograph and chip performance summary.

testing platform, shown in Fig. 14, consists of a host computer, an FPGA board, a dc power supply, an oscilloscope, and a PCB. The PCB incorporates a current meter for precise and rapid power consumption measurements. The auxiliary modules for LLM deployment, including layer normalization, residual, and Softmax, are implemented on the FPGA. During operation, OA-CIM macro interacts with off-chip systems through an on-chip bus interface. Control instructions are sent from the FPGA host via the FPGA mezzanine connector (FMC) and stored in the on-chip instruction memory. The top controller fetches and decodes these instructions, coordinating read/write operations and triggering computations within the macro, ensuring synchronized and efficient execution of tasks.

Fig. 15 presents the die photograph of the fabricated chip and summarizes the test results. The prototype occupies $2.96 \times 2.31 = 6.84$ mm^2 and integrates a 512 kB custom 8T SRAM CIM macro along with 96 kB foundry-provided standard SRAM, supporting three operational modes, namely Outlier, INT4, and BF16. The Outlier mode computes 64×128 sets of MAC operations per cycle between weights (either INT4 or BF16) and activations (either INT4 or their BF16-converted equivalents), which is consistent with the proposed ONAQ algorithm. The INT4 mode computes 64×128 sets of MAC operations per cycle between INT4 weights and INT4

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.

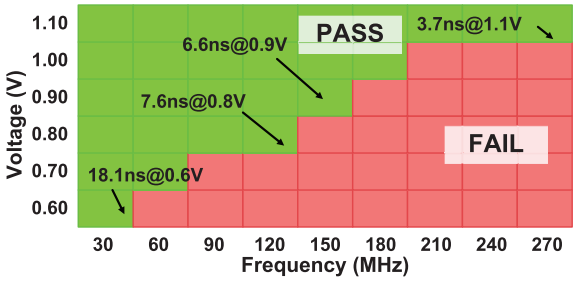
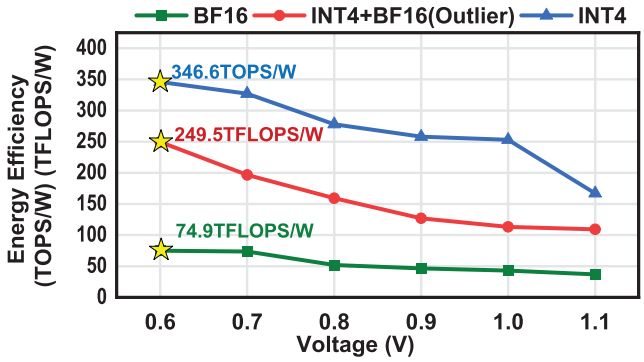


Fig. 16. Measured shmoo plot showing voltage versus working frequency.



*Measured peak energy efficiency with 90% input sparsity.

Fig. 17. Measured voltage-EEF scaling.

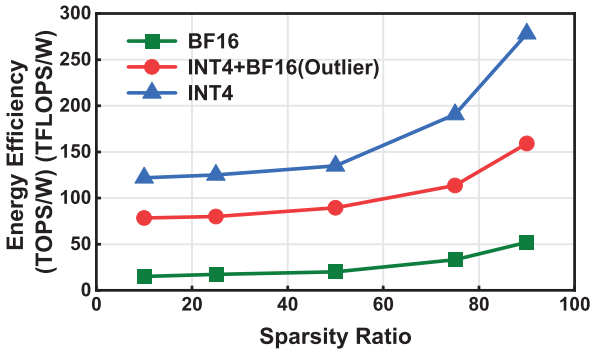


Fig. 18. Scaling of EEF with different sparsity levels.

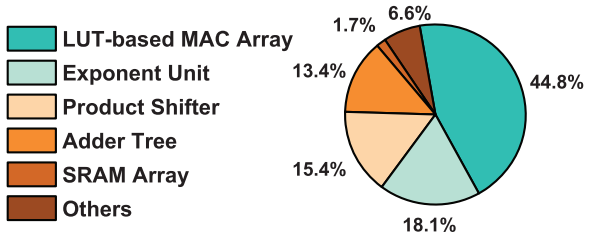


Fig. 19. Power breakdown of the proposed OA-CIM macro, obtained by post-layout simulation with Synopsys PrimeTime PX.

activations. In contrast, the BF16 mode computes 64 × 32 sets of MAC operations per cycle between BF16 weights and BF16 activations, aligning with standard FP-MAC operations. The three-mode design delivers optimal performance across scenarios: Outlier mode for efficient mixed-precision OA LLM inference, INT4 mode for high-throughput low-precision computation, and BF16 mode for full-precision model support and training.

According to the summary table, OA-CIM achieves a peak EEF of 249.5 TFLOPS/W@Outlier mode under 90% input sparsity, which is 3.33× higher than that in the full-BF16 mode. Meanwhile, the outlier mode results in only a 0.5% increase in perplexity for the OPT-6.7B and OPT-13B models compared to the full-BF16 case, while reducing the perplexity degradation by 10.4× relative to the INT4-only case. These results indicate that OA computation provides superior EEF and memory efficiency compared to full-BF16 without sacrificing model accuracy, thus offering a highly effective solution for efficient LLM deployment.

Fig. 16 presents the shmoo plot of the chip, which operates across a supply voltage range of 0.6 to 1.1 V, corresponding to frequency ranges from 55 to 270 MHz. The EEF scaling for the three operational modes over this voltage range is shown in Fig. 17. The highest EEF is achieved at 0.6 V and 55 MHz, where OA-CIM reaches 346.6 TOPS/W@INT4 mode, 74.9 TFLOPS/W@BF16 mode, and 249.5 TFLOPS/W@Outlier mode. The peak throughput of OA-CIM, measured at 1.1 V and 270 MHz, corresponds to 4.4 TOPS@INT4 mode, 1.1 TFLOPS@BF16 mode, and 4.4 TFLOPS@Outlier mode. It is important to note that the throughput at the Outlier mode reflects a mixed-precision computation involving both INT4 and BF16 MAC operations. Following established practices in prior works [30], [34], we express the throughput in TFLOPS for consistency. In this context, 1 OP is defined as one multiplication or one addition of BF16 or INT4 values. Furthermore, the reported Outlier mode throughput also reflects computational efficiency gains from ONAQ's weight pruning methodology, where operations associated with normal victim weights continue to contribute to the throughput measurement.

To better understand how EEF scales with sparsity, we provide measurements at different sparsity levels. Fig. 18 presents the results, conducted at 0.8 V and 130 MHz. Specifically, we evaluated EEF at sparsity levels of 10%, 25%, 50%, 75%, and 90%. The results demonstrate that EEF improves significantly as sparsity increases. For example, in the INT4 mode, EEF ranges from 122.5 TOPS/W at 10% sparsity to 278.1 TOPS/W at 90% sparsity. Similarly, in the outlier mode, EEF scales from 78.5 TFLOPS/W at 10% sparsity to 159.2 TFLOPS/W at 90% sparsity.

B. Evaluation of Techniques

In the following, we examine the power distribution across components in the OA-CIM macro and assess the contribution of each technique to the overall EEF.

Fig. 19 illustrates the power breakdown of the proposed OA-CIM macro based on post-layout simulation using Synopsys PrimeTime PX at 0.8 V and 130 MHz, excluding the peripheral control logic and input/output SRAM buffers. As shown, the MAC array contributes approximately 44.8% of the total power, while the exponent processing unit accounts for 18.1%. Other components, such as the product shifter and adder tree, consume 15.4% and 13.4% of the power, respectively.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON WITH STATE-OF-THE-ART CIM PROCESSORS/MACROS

	ISSCC'22[42]		ISSCC'23[48]	ISSCC'23[27]		ISSCC'23[34]		ISSCC'24[49]		ISSCC'24[47]		JSSC'24[50]	This Work		
Mode	INT/FP		FP	INT/FP		INT/FP		POSIT		FP		FP	**Outlier/INT/FP**		
Technology (nm)	28nm		22nm	28nm		28nm		28nm		22nm		28nm	**22nm**		
CIM Size	96Kb		832Kb	64Kb		64Kb		128Kb		16Mb		48Kb	**512Kb**		
Implementation	Digital SRAM		Hybrid SRAM	Digital SRAM		Digital SRAM		Digital SRAM		Analog ReRAM		Digital SRAM	**Digital SRAM**		
Supply Voltage(V)	0.6 - 1.0		0.6 - 0.8	0.6 - 0.9		0.4 - 0.9		0.55 - 1.0		0.7 - 0.8		0.5 - 1.3	**0.6 - 1.1**		
Access Time(ns)	4.5 - 20		6.4 (0.8V)	8.6 (0.8V)		2.5 - 100		2.4 - 12.8		4.8 - 6.0		3.3 - 80.0	**3.7- 18.1**		
Precision	INT8	BF16	BF16	INT8	BF16	INT4	FP16	P8	P16	BF16	FP16	FP16	**Outlier*2**	**INT4*3**	**BF16*4**
Energy Efficiency (TOPS/W) (TFLOPS/W)	57.8	46.2	70.2	44	31.6	300	16.9	55.6	27.6	65.5	60.4	53.3	**249.5**	**346.6**	**74.9**
Area Efficiency (TOPS/mm²) (TFLOPS/mm²)	0.2	0.16	0.069-0.071	2.5	2.0*5	0.36-2.12	NA	2.74	1.35	0.104	0.095	0.113	**2.06**	**2.06**	**0.52**
Application	BERT-Large(340M) @Nature Questions		CNN @CIFAR-100	CNN @CIFAR100		CNN @ImageNet		GPT-2 (762M) @(Wikitext-2)		CNN @ImageNet		MAC @KITTI	**LLM @(Wikitext-2, C4, PTB) OPT-1.3B, 2.7B, 6.7B, 13B**		

*1. Measured peak energy efficiency with 90% input sparsity. *2. In outlier mode, *1 **OP*** is defined as one ***multiplication*** or one ***addition*** of ***BF16*** or ***INT4*** value. *3. In INT4 mode, *1 **OP*** is defined as one ***multiplication*** or one ***addition*** of ***INT4*** value. *4. In BF16 mode, *1 **OP*** is defined as one ***multiplication*** or one ***addition*** of ***BF16*** value. *5. Exponent processing and normalization are excluded.

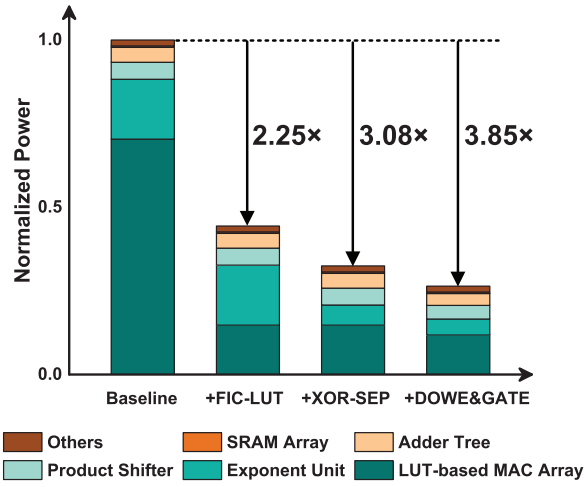


Fig. 20. Technique breakdown analysis on OPT inference benchmark.

Fig. 20 demonstrates the technique breakdown analysis on the OPT inference benchmark, clearly illustrating how OA-CIM's three key features contribute to system EEF improvements. We implemented a mixed-precision CIM baseline with Radix-4 Booth encoding, as described in [42]. By optimizing the MAC units with the FIC-LUT scheme, we reduce redundant addition operations, achieving a 2.25× improvement in EEF over the baseline. By sharing XOR-gate among the exponent unit and gating non-maximum exponent terms, the XOR-SEP scheme further improves EEF to 3.08× the baseline. Finally, the integration of the DOWE scheme, combined with dynamic-range-driven pre-charge gating, reduces readout power and minimizes toggling in the computation circuits. This results in a final EEF improvement of 3.85× over the baseline.

C. Comparison With State-of-the-Art Works

Table II presents a comparison between OA-CIM and several advanced CIM macros/processors [27], [34], [42], [47], [48], [49], [50]. This work introduces the first CIM macro supporting OA computation, while achieving a 2.7× to 3.1× improvement in EEF relative to prior mixed-precision designs [30], [34]. We use two figures of merit (FoM) to compare the performance. Using FoM_1, defined as IN Precision $\times W$ Precision × Output Ratio × Peak EEF to represent EEF normalized per bit, OA-CIM achieves a 1.57× improvement over [27], [34], [42] for INT-MAC operations and a 2.02× increase over [27], [34], [42], [47], [50] for FP-MAC operations on average. These improvements highlight the effectiveness of the INT/FP-compatible LUT MAC circuit in enhancing computational efficiency and the power optimization realized through the sparse readout scheme with weight encoding. Meanwhile, FoM_2, defined as IN Precision $\times W$ Precision × Output Ratio × Area Efficiency, reflects normalized computational density. Under this metric, OA-CIM exhibits a 2.6× improvement in INT-MAC operations over [42] and a 5.14× increase in FP-MAC operations on average compared to [42], [47], [48], [50], primarily attributed to the area efficiency of the XOR-gate-shared exponent circuit and the high utilization of MAC circuits across multiple modes.

VI. CONCLUSION

This work introduces OA-CIM, an OA digital SRAM CIM macro tailored for efficient LLM inference. The FIC LUT-based MAC circuit facilitates hybrid processing of BF16 outliers alongside INT4 normal values, ensuring accuracy while improving EEF. The XOR-sharing mechanism minimizes redundant circuits and bypasses unnecessary comparisons, contributing to lower latency and reduced hardware complexity. The DOWE method capitalizes on the nature of LLM

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.

weights, minimizing bitline toggling and reducing energy consumption in the readout process. Fabricated in a 22 nm CMOS technology, OA-CIM achieves an EEF of 346.6 TOPS/W in INT4 mode and 249.5 TFLOPS/W in outlier mode, representing a 2.7× to 3.1× efficiency gain over existing mixed-precision CIMs. These advancements position OA-CIM as a robust solution for deploying LLMs in resource-constrained environments.

REFERENCES

[1] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[2] S. Zhang et al., "OPT: Open pre-trained transformer language models," 2022, *arXiv:2205.01068*.

[3] OpenAI et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.

[4] G. Team et al., "Gemini: A family of highly capable multimodal models," 2023, *arXiv:2312.11805*.

[5] Z. Liu et al., "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 10012–10022.

[6] A. Vaswani et al., "Attention is all you need," in *Proc. NIPS*, 2017, pp. 6000–6010.

[7] M. V. Koroteev, "BERT: A review of applications in natural language processing and understanding," 2021, *arXiv:2103.11943*.

[8] Y. Qin et al., "MECLA: Memory-compute-efficient LLM accelerator with scaling sub-matrix partition," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, Jul. 2024, pp. 1032–1047.

[9] C. Tang, X. Lin, Z. Huang, W. Sun, H. Jia, and Y. Liu, "A 28nm 4.35TOPS/mm^2 transformer accelerator with basis-vector based ultra storage compression, decomposed computation and unified LUT-assisted cores," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2024, pp. 1–2.

[10] Y. Qin et al., "FACT: FFN-attention co-optimized transformer architecture with eager correlation prediction," in *Proc. ACM/IEEE Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2023, pp. 1–14.

[11] W.-S. Khwa et al., "A 65nm 4Kb algorithm-dependent computing-in-memory SRAM unit-macro with 2.3ns and 55.8TOPS/W fully parallel product-sum operation for binary DNN edge processors," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2018, pp. 496–498.

[12] Q. Dong et al., "15.3 A 351TOPS/W and 372.4GOPS compute-in-memory SRAM macro in 7nm FinFET CMOS for machine-learning applications," in *Proc. IEEE Int. Solid-State Circuits Conf.*, Feb. 2020, pp. 242–244.

[13] Q. Liu et al., "A fully integrated analog ReRAM based 78.4TOPS/W compute-in-memory chip with fully parallel MAC computing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 500–502.

[14] J. Wang et al., "A compute SRAM with bit-serial integer/floating-point operations for programmable in-memory vector acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2019, pp. 224–226.

[15] C.-X. Xue et al., "A 1Mb multibit ReRAM computing-in-memory macro with 14.6ns parallel MAC computing time for CNN based AI edge processors," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2019, pp. 388–390.

[16] M. Chang et al., "A 40nm 60.64TOPS/W ECC-capable compute-in-memory/digital 2.25MB/768KB RRAM/SRAM system with embedded cortex M3 microprocessor for edge recommendation systems," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[17] Y.-D. Chih et al., "An 89TOPS/W and 16.3TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 252–254.

[18] J. Yue et al., "15.2 A 2.75-to-75.9TOPS/W computing-in-memory NN processor supporting set-associate block-wise zero skipping and ping-pong CIM with simultaneous computation and weight updating," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 238–240.

[19] H. Diao et al., "A multiply-less approximate SRAM compute-in-memory macro for neural-network inference," *IEEE J. Solid-State Circuits*, vol. 60, no. 2, pp. 695–706, Feb. 2025.

[20] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," 2021, *arXiv:2103.13630*.

[21] C. Guo et al., "OliVe: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proc. 50th Annu. Int. Symp. Comput. Archit.*, Jun. 2023, pp. 1–15.

[22] Y. Bondarenko, M. Nagel, and T. Blankevoort, "Quantizable transformers: Removing outliers by helping attention heads do nothing," in *Proc. Int. Conf. Neural Inf. Process. Syst. (NIPS)*, Jun. 2023, pp. 1–21.

[23] J. Lee, W. Lee, and J. Sim, "Tender: Accelerating large language models via tensor decomposition and runtime requantization," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, Jul. 2024, pp. 1048–1062.

[24] C. Lee, J. Jin, T. Kim, H. Kim, and E. Park, "OWQ: Outlier-aware weight quantization for efficient fine-tuning and inference of large language models," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Mar. 2024, vol. 38, no. 12, pp. 13355–13364.

[25] T. Dettmers et al., "SpQR: A sparse-quantized representation for near-lossless LLM weight compression," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Jun. 2023, pp. 1–29.

[26] X. Wei et al., "Outlier suppression: Pushing the limit of low-bit transformer language models," in *Proc. Int. Conf. Neural Inf. Process. Syst. (NIPS)*, Jan. 2022, pp. 17402–17414.

[27] A. Guo et al., "A 28nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 128–130.

[28] W.-S. Khwa et al., "A 16nm 96Kb integer/floating-point dual-mode-gain-cell-computing-in-memory macro achieving 73.3-163.3TOPS/W and 33.2–91.2TFLOPS/W for AI-edge devices," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 67, Feb. 2024, pp. 568–570.

[29] Y. Yuan et al., "A 28nm 72.12TFLOPS/W hybrid-domain outer-product based floating-point SRAM computing-in-memory macro with logarithm bit-width residual ADC," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 67, Feb. 2024, pp. 576–578.

[30] R. Guo et al., "A 28nm 74.34TFLOPS/W BF16 heterogenous CIM-based accelerator exploiting denoising-similarity for diffusion models," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 67, Feb. 2024, pp. 362–364.

[31] S. Yan et al., "A 28-nm floating-point computing-in-memory processor using intensive-CIM sparse-digital architecture," *IEEE J. Solid-State Circuits*, vol. 59, no. 8, pp. 2630–2643, Aug. 2024.

[32] C.-F. Lee et al., "A 12nm 121-TOPS/W 41.6-TOPS/mm^2 all digital full precision SRAM-based compute-in-memory with configurable bit-width for AI edge applications," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2022, pp. 24–25.

[33] S. Jeong, J. Park, and D. Jeon, "A 28nm 1.644TFLOPS/W floating-point computation SRAM macro with variable precision for deep neural network inference and training," in *Proc. IEEE 48th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2022, pp. 145–148.

[34] J. Yue et al., "A 28nm 16.9–300TOPS/W computing-in-memory processor supporting floating-point NN inference/training with intensive-CIM sparse-digital architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 1–3.

[35] J. Lee et al., "A 13.7 TFLOPS/W floating-point DNN processor using heterogeneous computing architecture with exponent-computing-in-memory," in *Proc. Symp. VLSI Circuits*, Jun. 2021, pp. 1–2.

[36] H. Fujiwara et al., "A 3nm, 32.5TOPS/W, 55.0TOPS/mm^2 and 3.78Mb/mm^2 fully-digital compute-in-memory macro supporting INT12×INT12 with a parallel-MAC architecture and foundry 6T-SRAM bit cell," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 572–574.

[37] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," 2022, *arXiv:2210.17323*.

[38] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," 2016, *arXiv:1609.07843*.

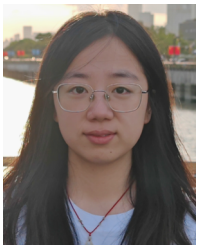
[39] M. Marcus et al., "The Penn treebank: Annotating predicate argument structure," in *Proc. Workshop Human Lang. Technol.*, 1994, pp. 114–119.

[40] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," 2019, *arXiv:1910.10683*.

[41] Ieee Standard for Floating-point Arithmetic, Standard 754–2008, Jul. 2019, pp. 1–84.

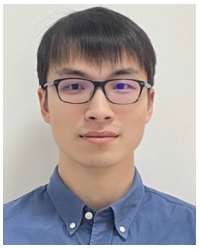
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.

[42] F. Tu et al., "A 28nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.
[43] H. Wu et al., "A 28-nm 19.9-to-258.5-TOPS/W 8b digital computing-in-memory processor with two-cycle macro featuring winograd-domain convolution and macro-level parallel dual-side sparsity," *IEEE J. Solid-State Circuits*, vol. 60, no. 1, pp. 347–361, Jan. 2025.
[44] A. Agrawal, "A 7nm 4-Core AI chip with 25.6TFLOPS hybrid FP8 training, 102.4TOPS INT4 inference and workload-aware throttling," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 144–146.
[45] H. Zhu et al., "COMB-MCM: Computing-on-memory-boundary NN processor with bipolar bitwise sparsity optimization for scalable multi-chiplet-module edge machine learning," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.
[46] H. Xia, "Quant-LLM: Accelerating the serving of large language models via FP6-centric algorithm-system co-design on modern GPUs," in *Proc. USENIX Secur. Symp.*, Jul. 2024, pp. 699–713.
[47] T.-H. Wen et al., "A 22nm 16Mb floating-point ReRAM compute-in-memory macro with 31.2TFLOPS/W for AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 67, Feb. 2024, pp. 580–582.
[48] P.-C. Wu et al., "A 22nm 832Kb hybrid-domain floating-point SRAM in-memory-compute macro with 16.2–70.2TFLOPS/W for high-accuracy AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 126–128.
[49] Y. Wang et al., "A 28nm 83.23TFLOPS/W POSIT-based compute-in-memory macro for high-accuracy AI applications," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 67, Feb. 2024, pp. 566–568.
[50] M. Li et al., "SLAM-CIM: A visual SLAM backend processor with dynamic-range-driven-skipping linear-solving FP-CIM macros," *IEEE J. Solid-State Circuits*, vol. 59, no. 11, pp. 3853–3865, Nov. 2024.



Siqi He (Graduate Student Member, IEEE) received the B.E. degree in microelectronics from Fudan University, Shanghai, China, in 2022, where she is currently pursuing the Ph.D. degree with the School of Microelectronics.

Her research interests include compute-in-memory and large language models.



Haozhe Zhu (Member, IEEE) received the B.E. and Ph.D. degrees in microelectronics from the School of Microelectronics, Fudan University, Shanghai, China, in 2018 and 2022, respectively.

From 2022 to 2025, he was a Post-Doctoral Research Fellow at Frontier Institute of Chip and System, Fudan University, where he became an Assistant Professor, in 2025. His current research interests include efficient integrated circuits and systems designed for intelligent applications.

Dr. Zhu is a co-awardee of the Distinguished Design Award from IEEE A-SSCC 2024.



Hongyi Zhang (Graduate Student Member, IEEE) received the B.E. degree in microelectronic science and engineering from Fudan University, Shanghai, China, in 2022, where she is currently pursuing the M.S. degree with the School of Microelectronics.

Her research interests include compute-in-memory and deep neural network circuit designs.



Yujie Ma received the B.S. degree in microelectronics science and engineering from Fudan University, Shanghai, China, in 2024, where he is currently pursuing the Ph.D. degree with the College of Integrated Circuits and Micro-Nano Electronics.

His research interests include digital CIM macro design for transformer.



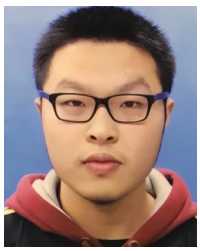
Zexing Chen received the B.S. degree in microelectronics science and engineering from Shenzhen University, Shenzhen, China, in 2023. He is currently pursuing the M.S. degree with Fudan University, Shanghai, China.

His research interests include memory peripheral circuit designs.



Mengjie Li (Graduate Student Member, IEEE) received the B.S. degree in electronic science and technology from Southeast University, Nanjing, China, in 2021. She is currently pursuing the Ph.D. degree with the School of Microelectronics, Fudan University, Shanghai, China.

Her research interests include compute-in-memory system design and domain-specific accelerators.



Danfeng Zhai (Graduate Student Member) received the B.E. degree in microelectronics from Fudan University, Shanghai, China, in 2020, where he is currently pursuing the Ph.D. degree.

His current research interests include calibration techniques for high-performance data converters and computing-in-memory circuits designs.



Chixiao Chen (Member, IEEE) received the B.S. and Ph.D. degrees in microelectronics from Fudan University, Shanghai, China, in 2010 and 2015, respectively.

In 2015, he worked at Calterah Inc., Shanghai, as an Analog/Mixed Signal Circuit Design Engineer. From 2016 to 2018, he was a Post-Doctoral Research Associate at the University of Washington, Seattle, WA, USA. In 2019, he joined Fudan University, where he is currently an Associate Professor with Frontier Institute of Circuits and Systems, and also the Director of the Integrated Chips Innovation Center, State Key Laboratory of Integrated Chips and Systems. His current research interests include mixed-signal integrated circuit design and custom intelligent software-hardware co-designs.

Dr. Chen has served as a TPC Member for A-SSCC since 2023.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.



Qi Liu (Member, IEEE) received the Ph.D. degree from Anhui University, Anhui, China, in 2010.

He then joined the Institute of Microelectronics, Chinese Academy of Sciences (IMECAS), Beijing, China, as a Professor, in 2016. He is currently a Professor with Frontier Institute of Chip and System, State Key Laboratory of Integrated Chips and Systems, Fudan University, Shanghai, China, and a Guest Professor with the IMECAS. His research interests include fabrication, characterization, and mechanisms of emerging memory devices and chips for memory, logic, computing-in-memory, and neuromorphic computing.



Xiaoyang Zeng (Member, IEEE) received the B.S. degree from Xiangtan University, Xiangtan, China, in 1992, and the Ph.D. degree from Changchun Institute of Optics, Fine Mechanics, and Physics, Chinese Academy of Sciences, Changchun, China, in 2001.

From 2001 to 2003, he was a Post-Doctoral Researcher with Fudan University, Shanghai, China. Then, he joined the State Key Laboratory of ASIC and System, Fudan University, as an Associate Professor, where he is currently a Full Professor. His research interests include information security chip design, system-on-chip platforms, and VLSI implementation of digital signal processing and communication systems.



Ming Liu (Fellow, IEEE) received the Ph.D. degree in material engineering from Beijing University of Aeronautics and Astronauts, Beijing, China, in 1998.

She joined the Institute of Microelectronics, Chinese Academy of Sciences, Beijing, in 1999, where she became a Full Professor, in 2000. She was selected as an Academician of the Chinese Academy of Sciences, Beijing, in 2015. She was the Director of the Key Laboratory of Microelectronics Devices and Integrated Technology, Institute of Microelectronics, Chinese Academy of Sciences. She is currently the Director of the Frontier Institute of Chip and System, State Key Laboratory of Integrated Chips and Systems, Fudan University, Shanghai, China. Her research interests include advanced semiconductor processes, emerging memory and logic technologies, novel integration and package technology, and next-generation computing paradigms.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:20 UTC from IEEE Xplore. Restrictions apply.